Introduction to Generative AI

Comprehensive Study Notes

Covering: Machine Learning Foundations · Neural Networks NLP · Transformers · LLMs · Prompt Engineering · RAG · Responsible AI

Topics Include:

Supervised & Unsupervised Learning · Decision Trees · Neural Networks
NLP & Word Embeddings · Transformer Architecture · Self-Attention
LLM Tooling · Prompt Engineering & Fine-Tuning · RAG Pipelines
Responsible AI & Bias Mitigation

Generated with complete code examples in Python

0 Contents

1 Foundations of Machine Learning **2**
1.1 Introduction to Machine Learning . . . 2
1.1.1 ML vs. Traditional Programming . . . 2
1.1.2 Difference Between AI, ML, and DL . . . 2
1.1.3 Types of Learning . . . 3
1.2 Types of ML Tasks . . . 3
1.2.1 Classification . . . 3
1.2.2 Regression . . . 3
1.2.3 Clustering . . . 3

2 Data Preprocessing **5**
2.1 Handling Missing Values . . . 5
2.2 Feature Scaling . . . 6
2.3 Data Encoding Methods . . . 6

3 Supervised Learning: Linear & Logistic Models **8**
3.1 Linear Regression . . . 8
3.1.1 Assumptions . . . 8
3.1.2 Model Formulation . . . 8
3.1.3 Limitations of Linear Regression . . . 9
3.2 Logistic Regression . . . 9
3.2.1 Model Formulation . . . 9
3.2.2 Limitations of Logistic Regression . . . 10

4 Machine Learning Model Evaluation **11**
4.1 Bias and Variance . . . 11
4.2 Classification Metrics . . . 11
4.2.1 Confusion Matrix . . . 11
4.2.2 Precision, Recall, and F1 Score . . . 12
4.3 Regression Metrics . . . 12
4.4 Cross-Validation . . . 13

5 Decision Trees **14**
5.1 Introduction to Decision Trees . . . 14
5.2 Impurity Measures: Gini Index . . . 14
5.3 Information Gain . . . 14
5.4 Advantages vs. Disadvantages . . . 15
5.5 Regression Trees . . . 15

6 Introduction to Neural Networks **17**
6.1 Architectures . . . 17
6.1.1 Single Cell Perceptron . . . 17
6.1.2 Single Layer Perceptron (SLP) . . . 17
6.1.3 Multi-Layer Perceptron (MLP) . . . 17
6.2 Training and Components . . . 18
6.2.1 Backpropagation . . . 18

6.2.2 Activation Functions . . . 19
6.2.3 Recurrent Neural Network (RNN) . . . 19

7 Natural Language Processing (NLP) 21
7.1 Text Preprocessing & Tokenization . . . 21
7.2 Word Embeddings . . . 21
7.3 Sequential Modeling . . . 23
7.4 Language Foundations . . . 23

8 LLM Architecture & Responsible AI 25
8.1 Transformers . . . 25
8.1.1 Self-Attention Mechanism . . . 25
8.2 Ethical Considerations . . . 27
8.2.1 Responsible AI & Fairness Principles . . . 27
8.2.2 Bias Mitigation . . . 28
8.2.3 Misinformation Prevention . . . 28

9 Transformers (Deep Dive) 30
9.1 Architecture Overview . . . 30

10 Generative AI & LLM Tooling Ecosystem 32
10.1 Overview . . . 32

11 Commercial APIs 33
11.1 OpenAI API . . . 33
11.2 Anthropic Claude API . . . 33

12 Prompt Engineering and Fine-Tuning 35
12.1 Prompt Engineering . . . 35
12.2 Fine-Tuning with Parameter-Efficient Methods (LoRA) . . . 36

13 RAG Agents: Pipelines, Embeddings & Vector Databases 38
13.1 What is RAG? . . . 38
13.2 Embedding Models . . . 38
13.3 Vector Databases . . . 38
13.4 Complete RAG Pipeline . . . 39

14 Responsible Use of AI 42
14.1 Core Principles . . . 42
14.2 Explainability with SHAP and LIME . . . 42
14.3 Data Privacy and Differential Privacy . . . 43
14.4 AI Safety: Output Guardrails . . . 44
14.5 Environmental Impact . . . 45

1 Foundations of Machine Learning

1.1 Introduction to Machine Learning

1.1.1 ML vs. Traditional Programming

Key Concept

Traditional Programming: Humans write explicit rules. Input + Rules → Output.

Machine Learning: The machine learns rules from data. Input + Output → Rules (Model).

In traditional programming, a developer explicitly codes every decision path. In ML, the algorithm learns patterns from historical data and generalises to unseen examples.

```python
# Traditional Programming - explicit rules
def classify_spam(email):
    if "free money" in email.lower():
        return "spam"
    if "lottery" in email.lower():
        return "spam"
    return "not spam"

# Machine Learning - data-driven rules
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer

vectorizer = CountVectorizer()
clf = MultinomialNB()

# Training
X_train_vec = vectorizer.fit_transform(X_train_emails)
clf.fit(X_train_vec, y_train_labels)  # learns from examples

# Prediction
X_test_vec = vectorizer.transform(X_test_emails)
predictions = clf.predict(X_test_vec)
```

Listing 1: Traditional vs ML Paradigm

1.1.2 Difference Between AI, ML, and DL

- **Artificial Intelligence (AI):** The broad field of making machines simulate human intelligence. Includes rule-based systems, expert systems, and ML.
- **Machine Learning (ML):** A subset of AI where systems learn from data without being explicitly programmed for every scenario.
- **Deep Learning (DL):** A subset of ML that uses multi-layered neural networks (deep neural nets) to learn hierarchical representations.

$$\text{AI} \supset \text{ML} \supset \text{Deep Learning}$$

1.1.3 Types of Learning

1. Supervised Learning: The model learns from labelled data (x_i, y_i). The goal is to learn a mapping $f : X \to Y$.

2. Unsupervised Learning: Data has no labels. The model discovers hidden structure (clusters, distributions) in $\{x_i\}$.

3. Semi-Supervised Learning: A small amount of labelled data combined with a large amount of unlabelled data.

4. Reinforcement Learning: An agent interacts with an environment, takes actions, and receives rewards or penalties to learn an optimal policy π.

```python
# Supervised - Labelled data
from sklearn.tree import DecisionTreeClassifier
clf = DecisionTreeClassifier()
clf.fit(X_train, y_train)       # y_train = labels
y_pred = clf.predict(X_test)

# Unsupervised - No labels
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X)                    # No y passed
labels = kmeans.labels_

# Semi-Supervised
from sklearn.semi_supervised import LabelPropagation
# -1 denotes unlabelled samples
y_mixed = [-1, -1, 0, 1, -1, 0, 1, -1]
lp = LabelPropagation()
lp.fit(X, y_mixed)
```

Listing 2: Types of Learning in sklearn

1.2 Types of ML Tasks

1.2.1 Classification

Predict a discrete class label $y \in \{c_1, c_2, \ldots, c_k\}$. Example: spam/not-spam, digit recognition.

1.2.2 Regression

Predict a continuous value $y \in \mathbb{R}$. Example: house price prediction, stock forecasting.

1.2.3 Clustering

Group data points into clusters without labels. Example: customer segmentation, document grouping.

```python
import numpy as np
from sklearn.datasets import make_classification, make_regression,
    make_blobs
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.cluster import KMeans

```

```python
# --- Classification ---
X_cls, y_cls = make_classification(n_samples=200, n_features=5, random_state=42)
lr_cls = LogisticRegression()
lr_cls.fit(X_cls, y_cls)
print("Classification accuracy:", lr_cls.score(X_cls, y_cls))

# --- Regression ---
X_reg, y_reg = make_regression(n_samples=200, n_features=3, noise=10, random_state=42)
lr_reg = LinearRegression()
lr_reg.fit(X_reg, y_reg)
print("Regression R2:", lr_reg.score(X_reg, y_reg))

# --- Clustering ---
X_clust, _ = make_blobs(n_samples=300, centers=4, random_state=42)
km = KMeans(n_clusters=4, random_state=42)
km.fit(X_clust)
print("Cluster centres:\n", km.cluster_centers_)
```

Listing 3: Classification

2 Data Preprocessing

2.1 Handling Missing Values

Missing data can arise due to data entry errors, sensor failures, or survey non-responses. Common strategies:

- **Deletion:** Remove rows/columns with missing values (only if missingness is small and random).
- **Mean/Median/Mode Imputation:** Replace with central tendency statistics.
- **KNN Imputation:** Use nearest neighbours to estimate missing values.
- **Iterative Imputation:** Model each feature with missing values as a function of other features.

```python
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer, KNNImputer

df = pd.DataFrame({
    'age':    [25, np.nan, 35, 40, np.nan],
    'salary': [50000, 60000, np.nan, 80000, 55000],
    'city':   ['NY', 'LA', np.nan, 'NY', 'LA']
})

# 1. Check missing values
print(df.isnull().sum())

# 2. Drop rows with any missing value
df_dropped = df.dropna()

# 3. Mean imputation for numerical columns
imputer_mean = SimpleImputer(strategy='mean')
df[['age', 'salary']] = imputer_mean.fit_transform(df[['age', 'salary
    ']])

# 4. Mode imputation for categorical columns
imputer_mode = SimpleImputer(strategy='most_frequent')
df[['city']] = imputer_mode.fit_transform(df[['city']])

# 5. KNN Imputation
knn_imp = KNNImputer(n_neighbors=2)
df_knn = pd.DataFrame(
    knn_imp.fit_transform(df[['age', 'salary']]),
    columns=['age', 'salary']
)
print(df_knn)
```

Listing 4: Handling Missing Values

2.2 Feature Scaling

Many ML algorithms (e.g., SVMs, k-NN, gradient descent) are sensitive to the scale of features.

Formula

Min-Max Scaling (Normalisation):

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Standardisation (Z-score):

$$x' = \frac{x - \mu}{\sigma}$$

```python
from sklearn.preprocessing import MinMaxScaler, StandardScaler,
    RobustScaler

X = [[100, 0.001],
     [200, 0.002],
     [300, 0.003],
     [150, 0.0015]]

# Min-Max Scaling (output in [0, 1])
minmax = MinMaxScaler()
X_minmax = minmax.fit_transform(X)
print("Min-Max:\n", X_minmax)

# Standardisation (mean=0, std=1)
standard = StandardScaler()
X_standard = standard.fit_transform(X)
print("Standardised:\n", X_standard)

# Robust Scaler (uses median and IQR - robust to outliers)
robust = RobustScaler()
X_robust = robust.fit_transform(X)
print("Robust:\n", X_robust)
```

Listing 5: Feature Scaling Methods

2.3 Data Encoding Methods

Categorical variables must be converted to numerical form for ML algorithms.

```python
import pandas as pd
from sklearn.preprocessing import LabelEncoder, OrdinalEncoder,
    OneHotEncoder
import numpy as np

df = pd.DataFrame({'color': ['Red','Blue','Green','Red','Blue'],
                   'size':  ['S','M','L','XL','M'],
                   'price': [10, 20, 30, 40, 50]})

# 1. Label Encoding (ordinal semantics - use carefully)
```

```python
le = LabelEncoder()
df['color_encoded'] = le.fit_transform(df['color'])
print(df['color_encoded'].values)   # [2, 0, 1, 2, 0]

# 2. One-Hot Encoding (no ordinal relationship assumed)
ohe = OneHotEncoder(sparse_output=False)
color_ohe = ohe.fit_transform(df[['color']])
print(color_ohe)

# With pandas get_dummies
df_dummies = pd.get_dummies(df, columns=['color'], drop_first=True)
print(df_dummies.head())

# 3. Ordinal Encoding (when order matters: S < M < L < XL)
oe = OrdinalEncoder(categories=[['S','M','L','XL']])
df['size_ordinal'] = oe.fit_transform(df[['size']])
print(df['size_ordinal'].values)    # [0, 1, 2, 3, 1]
```

Listing 6: Encoding Categorical Variables

3 Supervised Learning: Linear & Logistic Models

3.1 Linear Regression

3.1.1 Assumptions

1. **Linearity:** The relationship between predictors and response is linear.
2. **Independence:** Observations are independent of each other.
3. **Homoscedasticity:** Constant variance of residuals.
4. **Normality:** Residuals are normally distributed.
5. **No multicollinearity:** Predictors are not highly correlated with each other.

3.1.2 Model Formulation

Formula

For p features:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p = \mathbf{X}\boldsymbol{\beta}$$

Ordinary Least Squares (OLS) Solution:

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Cost Function (MSE):

$$J(\boldsymbol{\beta}) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

```python
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# ---------- From Scratch (OLS) ----------
np.random.seed(42)
X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X.squeeze() + np.random.randn(100)

X_b = np.c_[np.ones((100, 1)), X]            # add bias column
beta_hat = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
print(f"Intercept (OLS): {beta_hat[0]:.4f}")
print(f"Slope (OLS):     {beta_hat[1]:.4f}")

# ---------- Using sklearn ----------
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
    =0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```python
print(f"R2 Score: {r2_score(y_test, y_pred):.4f}")
print(f"MSE:      {mean_squared_error(y_test, y_pred):.4f}")
print(f"Coefficients: {model.coef_}, Intercept: {model.intercept_:.4f}")
```

Listing 7: Linear Regression from Scratch and with sklearn

3.1.3 Limitations of Linear Regression

- Assumes linear relationship — fails on non-linear data.
- Highly sensitive to outliers.
- Cannot handle classification tasks directly.
- Multicollinearity inflates coefficient variance.

3.2 Logistic Regression

3.2.1 Model Formulation

Logistic Regression models the probability of a binary outcome:

Formula

Sigmoid Function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Model:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\boldsymbol{\beta}^\top \mathbf{x}) = \frac{1}{1 + e^{-\boldsymbol{\beta}^\top \mathbf{x}}}$$

Log-Loss (Cross-Entropy):

$$J(\boldsymbol{\beta}) = -\frac{1}{n} \sum_{i=1}^{n} [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)]$$

The sigmoid maps any real-valued input $z \in (-\infty, +\infty)$ to a probability in $(0, 1)$.

```python
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report

# Sigmoid function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

z_vals = np.array([-5, -2, 0, 2, 5])
print("Sigmoid:", sigmoid(z_vals))  # [0.007, 0.119, 0.5, 0.880, 0.993]

# Logistic Regression with sklearn
```

```python
data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
    =0.2, random_state=42)

scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train_sc, y_train)

y_pred = model.predict(X_test_sc)
y_prob = model.predict_proba(X_test_sc)[:, 1]

print(classification_report(y_test, y_pred, target_names=data.
    target_names))
```

Listing 8: Logistic Regression Implementation

3.2.2 Limitations of Logistic Regression

- Assumes linear decision boundary — struggles with complex non-linear patterns.
- Requires feature engineering for non-linear relationships.
- Sensitive to class imbalance.
- Cannot natively handle multi-class without modifications (OvR, OvO).

4 Machine Learning Model Evaluation

4.1 Bias and Variance

Key Concept

Bias: Error from wrong assumptions (model too simple — underfitting).

Variance: Error from sensitivity to small fluctuations in training data (model too complex — overfitting).

Formula

Bias-Variance Decomposition:

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

```python
import numpy as np
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score

np.random.seed(0)
X = np.sort(np.random.rand(30, 1) * 6, axis=0)
y = np.sin(X).ravel() + np.random.randn(30) * 0.3

degrees = [1, 3, 10, 20]  # 1=underfitting, 20=overfitting
for d in degrees:
    model = make_pipeline(PolynomialFeatures(d), LinearRegression())
    scores = cross_val_score(model, X, y, cv=5, scoring='neg_mean_squared_error')
    mse = -scores.mean()
    std = scores.std()
    print(f"Degree {d:2d}: MSE = {mse:.4f} (+/- {std:.4f})")
# Degree  1: High bias, low variance (underfitting)
# Degree 20: Low bias, high variance (overfitting)
# Degree  3: Best tradeoff
```

Listing 9: Visualising Bias-Variance Tradeoff

4.2 Classification Metrics

4.2.1 Confusion Matrix

$$\begin{pmatrix} \text{TN} & \text{FP} \\ \text{FN} & \text{TP} \end{pmatrix}$$

- **TP** (True Positive): Correctly predicted positive.
- **TN** (True Negative): Correctly predicted negative.
- **FP** (False Positive): Predicted positive, actually negative (Type I error).

- **FN** (False Negative): Predicted negative, actually positive (Type II error).

4.2.2 Precision, Recall, and F1 Score

Formula

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall (Sensitivity)} = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

```python
from sklearn.metrics import (confusion_matrix, classification_report,
                             precision_score, recall_score, f1_score,
                             ConfusionMatrixDisplay)
import numpy as np

y_true = np.array([1, 0, 1, 1, 0, 1, 0, 0, 1, 0])
y_pred = np.array([1, 0, 1, 0, 0, 1, 1, 0, 1, 0])

cm = confusion_matrix(y_true, y_pred)
print("Confusion Matrix:\n", cm)

print(f"Precision: {precision_score(y_true, y_pred):.2f}")
print(f"Recall:    {recall_score(y_true, y_pred):.2f}")
print(f"F1 Score:  {f1_score(y_true, y_pred):.2f}")
print("\nFull Report:\n", classification_report(y_true, y_pred))
```

Listing 10: Classification Metrics

4.3 Regression Metrics

Formula

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i|$$

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \qquad \text{RMSE} = \sqrt{\text{MSE}}$$

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2} \qquad R^2_{\text{adj}} = 1 - \frac{(1 - R^2)(n - 1)}{n - p - 1}$$

```python
from sklearn.metrics import mean_absolute_error, mean_squared_error,
    r2_score
import numpy as np

y_true = np.array([3.5, 2.0, 5.5, 8.0, 4.0, 6.5])
```

```python
y_pred = np.array([3.0, 2.5, 5.0, 7.5, 4.5, 6.0])

mae  = mean_absolute_error(y_true, y_pred)
mse  = mean_squared_error(y_true, y_pred)
rmse = np.sqrt(mse)
r2   = r2_score(y_true, y_pred)

n, p = len(y_true), 1  # n samples, p features
r2_adj = 1 - (1 - r2) * (n - 1) / (n - p - 1)

print(f"MAE:          {mae:.4f}")
print(f"MSE:          {mse:.4f}")
print(f"RMSE:         {rmse:.4f}")
print(f"R2:           {r2:.4f}")
print(f"Adjusted R2:  {r2_adj:.4f}")
```

Listing 11: Regression Metrics

4.4 Cross-Validation

Cross-validation provides a robust estimate of model performance by using multiple train/test splits.

```python
from sklearn.model_selection import (KFold, StratifiedKFold,
                                     cross_val_score, cross_validate)
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris
import numpy as np

X, y = load_iris(return_X_y=True)
model = LogisticRegression(max_iter=1000)

# Standard K-Fold (k=5)
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=kf, scoring='accuracy')
print(f"K-Fold Accuracy: {scores.mean():.4f} +/- {scores.std():.4f}")

# Stratified K-Fold (maintains class distribution in each fold)
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores_s = cross_val_score(model, X, y, cv=skf, scoring='accuracy')
print(f"Stratified K-Fold: {scores_s.mean():.4f} +/- {scores_s.std():.4f}")

# Multiple metrics at once
results = cross_validate(model, X, y, cv=5,
                         scoring=['accuracy', 'f1_weighted'],
                         return_train_score=True)
print("Test Accuracy:",  results['test_accuracy'].mean())
print("Train Accuracy:", results['train_accuracy'].mean())
```

Listing 12: K-Fold and Stratified Cross-Validation

5 Decision Trees

5.1 Introduction to Decision Trees

A Decision Tree is a flowchart-like structure where each internal node represents a feature test, each branch represents a test outcome, and each leaf node represents a class label or value.

Key Concept

Decision Trees are non-parametric, white-box models that are easy to visualise and interpret. They work for both classification and regression.

5.2 Impurity Measures: Gini Index

Formula

Gini Impurity:

$$\text{Gini}(t) = 1 - \sum_{k=1}^{K} p_k^2$$

Where p_k is the proportion of class k at node t. A Gini of 0 means perfect purity (all samples belong to one class).

Entropy:

$$H(t) = -\sum_{k=1}^{K} p_k \log_2(p_k)$$

5.3 Information Gain

Formula

$$\text{IG}(t, A) = \text{Impurity}(t) - \sum_{v \in \text{values}(A)} \frac{|t_v|}{|t|} \cdot \text{Impurity}(t_v)$$

The feature A with the highest information gain is chosen as the splitting criterion.

```python
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np

iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)
```

```python

# Train Decision Tree
dt = DecisionTreeClassifier(
    criterion='gini',        # or 'entropy'
    max_depth=3,
    min_samples_split=5,
    min_samples_leaf=2,
    random_state=42
)
dt.fit(X_train, y_train)

# Evaluate
y_pred = dt.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")

# Text representation of the tree
print(export_text(dt, feature_names=iris.feature_names))

# Feature importances
for name, imp in zip(iris.feature_names, dt.feature_importances_):
    print(f"  {name}: {imp:.4f}")
```

Listing 13: Decision Tree Classifier and Visualisation

5.4 Advantages vs. Disadvantages

Advantages:

- Easy to interpret and visualise
- No feature scaling required
- Handles both numerical and categorical data
- Non-parametric (no distributional assumptions)
- Fast inference

Disadvantages:

- Prone to overfitting (high variance)
- Instability — small data changes create different trees
- Biased toward features with more levels
- Not optimal for continuous variables
- Poor extrapolation beyond training range

5.5 Regression Trees

```python
from sklearn.tree import DecisionTreeRegressor
from sklearn.datasets import make_regression
from sklearn.metrics import mean_squared_error
import numpy as np

X, y = make_regression(n_samples=200, n_features=1, noise=20,
    random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
    =0.2, random_state=42)

# Regression Tree - splits by minimising MSE in each leaf
reg_tree = DecisionTreeRegressor(max_depth=4, random_state=42)
reg_tree.fit(X_train, y_train)
y_pred = reg_tree.predict(X_test)

```

```python
mse  = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
print(f"RMSE: {rmse:.4f}")
# Leaf node predictions are the mean of training samples in that leaf
```

Listing 14: Decision Tree for Regression

6 Introduction to Neural Networks

6.1 Architectures

6.1.1 Single Cell Perceptron

The simplest neural unit: takes weighted inputs, sums them, applies a step activation.

Formula

$$\text{output} = \begin{cases} 1 & \text{if } \sum_i w_i x_i + b \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

6.1.2 Single Layer Perceptron (SLP)

A network with one layer of input neurons connected directly to output neurons. Can only learn linearly separable patterns.

6.1.3 Multi-Layer Perceptron (MLP)

An MLP has one or more hidden layers between input and output, enabling learning of non-linear decision boundaries.

Formula

For layer l with weights $\mathbf{W}^{(l)}$, biases $\mathbf{b}^{(l)}$, and activation g:

$$\mathbf{a}^{(l)} = g\big(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}\big)$$

```python
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import numpy as np

# Data
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

sc = StandardScaler()
X_train = torch.FloatTensor(sc.fit_transform(X_train))
X_test  = torch.FloatTensor(sc.transform(X_test))
y_train = torch.FloatTensor(y_train)
y_test  = torch.FloatTensor(y_test)

# Define MLP
class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super(MLP, self).__init__()
```

```python
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(hidden_dim, hidden_dim // 2),
            nn.ReLU(),
            nn.Linear(hidden_dim // 2, output_dim),
            nn.Sigmoid()
        )
    def forward(self, x):
        return self.net(x).squeeze()

model = MLP(input_dim=20, hidden_dim=64, output_dim=1)
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Training loop
for epoch in range(50):
    model.train()
    optimizer.zero_grad()
    outputs = model(X_train)
    loss = criterion(outputs, y_train)
    loss.backward()
    optimizer.step()
    if (epoch+1) % 10 == 0:
        print(f"Epoch {epoch+1:3d}, Loss: {loss.item():.4f}")
```

Listing 15: Building an MLP with PyTorch

6.2 Training and Components

6.2.1 Backpropagation

Backpropagation computes the gradient of the loss with respect to each weight using the chain rule, enabling gradient descent updates.

Formula

Chain Rule for Layer l:

$$\frac{\partial J}{\partial \mathbf{W}^{(l)}} = \frac{\partial J}{\partial \mathbf{a}^{(l)}} \cdot \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} \cdot \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{W}^{(l)}}$$

Gradient Descent Update:

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \alpha \frac{\partial J}{\partial \mathbf{W}^{(l)}}$$

where α is the learning rate.

6.2.2 Activation Functions

Formula

$$\text{Sigmoid: } \sigma(z) = \frac{1}{1+e^{-z}} \qquad \text{Range: } (0,1)$$

$$\text{Tanh: } \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \qquad \text{Range: } (-1,1)$$

$$\text{ReLU: } f(z) = \max(0,z) \qquad \text{Range: } [0,\infty)$$

$$\text{Leaky ReLU: } f(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0 \end{cases}$$

$$\text{Softmax: } \sigma(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}} \qquad \text{(Multi-class output)}$$

```python
import torch
import torch.nn.functional as F
import torch.nn as nn

z = torch.tensor([-2.0, -1.0, 0.0, 1.0, 2.0])

print("Sigmoid: ", torch.sigmoid(z))
print("Tanh:    ", torch.tanh(z))
print("ReLU:    ", F.relu(z))
print("Leaky:   ", F.leaky_relu(z, negative_slope=0.01))
print("GELU:    ", F.gelu(z))

# Softmax for multi-class
logits = torch.tensor([1.5, 0.5, -1.0, 2.0])
probs = F.softmax(logits, dim=0)
print("Softmax: ", probs)  # sums to 1.0

# In a model: choose activation based on task
# - Hidden layers: ReLU, GELU, Leaky ReLU
# - Binary classification output: Sigmoid
# - Multi-class output: Softmax
# - Regression output: Linear (no activation)
```

Listing 16: Activation Functions in PyTorch

6.2.3 Recurrent Neural Network (RNN)

RNNs process sequential data by maintaining a hidden state that is updated at each time step.

Formula

$$h_t = \tanh(W_{hh}\, h_{t-1} + W_{xh}\, x_t + b_h)$$

$$y_t = W_{hy}\, h_t + b_y$$

```python
import torch
import torch.nn as nn

class SimpleRNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size, num_layers=1):
        super(SimpleRNN, self).__init__()
        self.hidden_size = hidden_size
        self.num_layers  = num_layers
        self.rnn = nn.RNN(input_size, hidden_size, num_layers,
                          batch_first=True, nonlinearity='tanh')
        self.fc  = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        h0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size)
        out, hn = self.rnn(x, h0)
        out = self.fc(out[:, -1, :])  # last time step
        return out

# Input: batch=8, seq_len=10, features=5
model = SimpleRNN(input_size=5, hidden_size=32, output_size=1)
x = torch.randn(8, 10, 5)
output = model(x)
print("RNN output shape:", output.shape)  # (8, 1)

# LSTM (preferred over vanilla RNN - solves vanishing gradient)
lstm = nn.LSTM(input_size=5, hidden_size=32, num_layers=2,
               batch_first=True, dropout=0.2)
out, (hn, cn) = lstm(x)
print("LSTM output shape:", out.shape)   # (8, 10, 32)
```

Listing 17: RNN in PyTorch

7 Natural Language Processing (NLP)

7.1 Text Preprocessing & Tokenization

```python
import re
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from collections import Counter

# Download required resources (run once)
# nltk.download('punkt'); nltk.download('stopwords'); nltk.download('wordnet')

text = "Natural Language Processing (NLP) is fascinating! It enables machines to understand, interpret, and generate human language."

# 1. Lowercasing
text_lower = text.lower()

# 2. Remove punctuation and special characters
text_clean = re.sub(r'[^a-z\s]', '', text_lower)

# 3. Tokenisation
tokens = word_tokenize(text_clean)
print("Tokens:", tokens)

# 4. Stop word removal
stop_words = set(stopwords.words('english'))
tokens_no_stop = [t for t in tokens if t not in stop_words]
print("No stopwords:", tokens_no_stop)

# 5. Stemming (aggressive, may produce non-words)
stemmer = PorterStemmer()
stemmed = [stemmer.stem(t) for t in tokens_no_stop]
print("Stemmed:", stemmed)

# 6. Lemmatisation (dictionary-based, preserves meaning)
lemmatizer = WordNetLemmatizer()
lemmatized = [lemmatizer.lemmatize(t) for t in tokens_no_stop]
print("Lemmatized:", lemmatized)

# 7. Bag of Words
vocab = Counter(tokens_no_stop)
print("Top words:", vocab.most_common(5))
```

Listing 18: Text Preprocessing Pipeline

7.2 Word Embeddings

Word embeddings represent words as dense real-valued vectors, capturing semantic relationships.

Key Concept

Key property: $\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$

Embeddings encode semantic similarity: similar words have similar vectors (high cosine similarity).

```python
# --- Option 1: Train Word2Vec from scratch ---
from gensim.models import Word2Vec

corpus = [
    ["machine", "learning", "is", "powerful"],
    ["deep", "learning", "uses", "neural", "networks"],
    ["natural", "language", "processing", "understands", "text"],
    ["transformers", "revolutionised", "natural", "language", "processing"],
]

model_w2v = Word2Vec(corpus, vector_size=100, window=3, min_count=1, sg=1, epochs=100)
# sg=1 -> Skip-gram; sg=0 -> CBOW

vector = model_w2v.wv['learning']
print("Shape:", vector.shape)  # (100,)

similar = model_w2v.wv.most_similar('learning', topn=3)
print("Similar to 'learning':", similar)

# --- Option 2: Pre-trained embeddings from HuggingFace ---
from transformers import AutoTokenizer, AutoModel
import torch

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
bert_model = AutoModel.from_pretrained("bert-base-uncased")

text = "Machine learning is amazing"
inputs = tokenizer(text, return_tensors="pt", padding=True, truncation=True)

with torch.no_grad():
    outputs = bert_model(**inputs)

# CLS token embedding (sentence representation)
cls_embedding = outputs.last_hidden_state[:, 0, :]
print("BERT CLS embedding shape:", cls_embedding.shape)  # (1, 768)

# Mean pooling over all tokens
mean_embedding = outputs.last_hidden_state.mean(dim=1)
print("Mean pooled shape:", mean_embedding.shape)  # (1, 768)
```

Listing 19: Word Embeddings with Gensim and Transformers

7.3 Sequential Modeling

Sequential models process text token by token, maintaining context through hidden states or attention.

```python
import torch
import torch.nn as nn

class TextLSTM(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim, pad_idx):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=pad_idx)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, num_layers=2,
                            batch_first=True, dropout=0.3, bidirectional=True)
        self.dropout = nn.Dropout(0.4)
        self.fc = nn.Linear(hidden_dim * 2, output_dim)   # *2 for bidirectional

    def forward(self, text):
        embedded = self.dropout(self.embedding(text))   # (batch, seq, embed)
        out, (hidden, cell) = self.lstm(embedded)
        # Concatenate final forward and backward hidden states
        hidden_cat = torch.cat([hidden[-2], hidden[-1]], dim=1)
        return self.fc(self.dropout(hidden_cat))

# Instantiate
model = TextLSTM(vocab_size=10000, embed_dim=128,
                 hidden_dim=256, output_dim=2, pad_idx=0)
dummy_text = torch.randint(0, 10000, (32, 50))  # batch=32, seq_len=50
output = model(dummy_text)
print("Output shape:", output.shape)  # (32, 2) for binary classification
```

Listing 20: Text Classification with LSTM

7.4 Language Foundations

N-grams: Contiguous sequences of n tokens. Unigrams (n=1), bigrams (n=2), trigrams (n=3).

TF-IDF (Term Frequency – Inverse Document Frequency):

Formula

$$\mathrm{TF}(t,d) = \frac{\text{count of } t \text{ in } d}{\text{total tokens in } d}$$

$$\mathrm{IDF}(t) = \log\left(\frac{N}{1+df(t)}\right) \qquad \text{TF-IDF}(t,d) = \mathrm{TF}(t,d) \times \mathrm{IDF}(t)$$

```python
from sklearn.feature_extraction.text import TfidfVectorizer,
    CountVectorizer

corpus = [
    "The cat sat on the mat",
    "The dog sat on the mat",
    "The cat and the dog are friends"
]

# TF-IDF
tfidf = TfidfVectorizer(max_features=20, stop_words='english')
X_tfidf = tfidf.fit_transform(corpus)
print("TF-IDF shape:", X_tfidf.shape)
print("Features:", tfidf.get_feature_names_out())

# Bigram features
bigram = CountVectorizer(ngram_range=(2, 2))
X_bigram = bigram.fit_transform(corpus)
print("Bigrams:", bigram.get_feature_names_out())

# Character-level n-grams (useful for morphology)
char_ngram = TfidfVectorizer(analyzer='char_wb', ngram_range=(3, 4))
X_char = char_ngram.fit_transform(corpus)
print("Char n-gram shape:", X_char.shape)
```

Listing 21: TF-IDF and N-grams

8 LLM Architecture & Responsible AI

8.1 Transformers

The Transformer, introduced in the landmark 2017 paper "Attention is All You Need", replaced RNNs with a purely attention-based architecture. It enables parallel processing of sequences and captures long-range dependencies efficiently.

Core components:

1. **Tokenisation & Input Embedding** — Convert tokens to vectors.
2. **Positional Encoding** — Inject sequence order information.
3. **Multi-Head Self-Attention** — Relate each token to all others.
4. **Feed-Forward Network (FFN)** — Apply non-linear transformation per position.
5. **Layer Normalisation & Residual Connections** — Stabilise training.
6. **Encoder-Decoder or Decoder-only** — Architecture variant depends on task.

8.1.1 Self-Attention Mechanism

Formula

Scaled Dot-Product Attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Where:

- $Q \in \mathbb{R}^{n \times d_k}$ — Query matrix
- $K \in \mathbb{R}^{n \times d_k}$ — Key matrix
- $V \in \mathbb{R}^{n \times d_v}$ — Value matrix
- $\sqrt{d_k}$ — Scaling factor to prevent gradient saturation

Multi-Head Attention:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class SelfAttention(nn.Module):
    def __init__(self, embed_dim, num_heads):
        super().__init__()
        assert embed_dim % num_heads == 0
        self.d_k      = embed_dim // num_heads
        self.num_heads = num_heads
```

```python
        self.embed_dim = embed_dim

        self.W_q = nn.Linear(embed_dim, embed_dim)
        self.W_k = nn.Linear(embed_dim, embed_dim)
        self.W_v = nn.Linear(embed_dim, embed_dim)
        self.W_o = nn.Linear(embed_dim, embed_dim)

    def split_heads(self, x, batch_size):
        x = x.view(batch_size, -1, self.num_heads, self.d_k)
        return x.transpose(1, 2)  # (batch, heads, seq, d_k)

    def forward(self, x, mask=None):
        batch_size = x.size(0)
        Q = self.split_heads(self.W_q(x), batch_size)
        K = self.split_heads(self.W_k(x), batch_size)
        V = self.split_heads(self.W_v(x), batch_size)

        # Scaled dot-product attention
        scores = (Q @ K.transpose(-2, -1)) / math.sqrt(self.d_k)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, float('-inf'))
        attn_weights = F.softmax(scores, dim=-1)

        context = attn_weights @ V              # (batch, heads, seq, d_k)
        context = context.transpose(1, 2)        # (batch, seq, heads, d_k)
        context = context.contiguous().view(batch_size, -1, self.embed_dim)
        return self.W_o(context), attn_weights

# Test
attn = SelfAttention(embed_dim=512, num_heads=8)
x = torch.randn(2, 10, 512)  # batch=2, seq=10, dim=512
out, weights = attn(x)
print("Output:", out.shape)           # (2, 10, 512)
print("Attention Weights:", weights.shape)  # (2, 8, 10, 10)
```

Listing 22: Self-Attention from Scratch in PyTorch

```python
import torch
import torch.nn as nn
import math

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float()
                             * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)  # even dims
        pe[:, 1::2] = torch.cos(position * div_term)  # odd dims
        pe = pe.unsqueeze(0)                            # (1, max_len, d_model)
```

```python
        self.register_buffer('pe', pe)

    def forward(self, x):
        # x: (batch, seq_len, d_model)
        x = x + self.pe[:, :x.size(1), :]
        return self.dropout(x)

pe = PositionalEncoding(d_model=512)
x = torch.zeros(2, 10, 512)
out = pe(x)
print("PE output shape:", out.shape)  # (2, 10, 512)
```

Listing 23: Positional Encoding

```python
from transformers import pipeline, AutoTokenizer, AutoModelForSequenceClassification
import torch

# --- Sentiment Analysis ---
classifier = pipeline("sentiment-analysis",
                      model="distilbert-base-uncased-finetuned-sst-2-english")
result = classifier("I absolutely love this course on machine learning!")
print(result)  # [{'label': 'POSITIVE', 'score': 0.9997}]

# --- Text Generation ---
generator = pipeline("text-generation", model="gpt2")
output = generator("The future of AI is", max_length=50, num_return_sequences=1)
print(output[0]['generated_text'])

# --- Named Entity Recognition ---
ner = pipeline("ner", grouped_entities=True)
entities = ner("Elon Musk founded SpaceX in California in 2002.")
for e in entities:
    print(f"{e['word']:<20} -> {e['entity_group']}")
```

Listing 24: Using HuggingFace Transformers for Inference

8.2 Ethical Considerations

8.2.1 Responsible AI & Fairness Principles

Key Concept

Core Principles of Responsible AI:

1. **Fairness** — Systems should not discriminate against protected groups.
2. **Accountability** — Humans remain responsible for AI-driven decisions.
3. **Transparency** — Models and decisions should be explainable.
4. **Privacy** — User data must be protected and minimised.
5. **Safety** — Systems should not cause physical or social harm.
6. **Inclusivity** — Systems should work for all demographics.

Fairness Metrics:

Formula

$$\text{Demographic Parity: } P(\hat{Y} = 1 \mid A = 0) = P(\hat{Y} = 1 \mid A = 1)$$

$$\text{Equal Opportunity: } P(\hat{Y} = 1 \mid Y = 1, A = 0) = P(\hat{Y} = 1 \mid Y = 1, A = 1)$$

8.2.2 Bias Mitigation

```python
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix

# Simulate biased dataset
np.random.seed(42)
n = 1000
# Feature: income, Protected attribute: gender (0=Female, 1=Male)
gender    = np.random.randint(0, 2, n)
income    = np.random.normal(50000 + gender * 10000, 15000, n)
# Biased label: hiring (males more likely due to bias in historical data)
hire_prob = 1 / (1 + np.exp(-(0.00003 * income + 0.5 * gender - 3)))
hired     = np.random.binomial(1, hire_prob, n)

X = np.column_stack([income, gender])
model = LogisticRegression()
model.fit(X, hired)
y_pred = model.predict(X)

# Check demographic parity
hire_rate_female = y_pred[gender == 0].mean()
hire_rate_male   = y_pred[gender == 1].mean()
print(f"Hire rate Female: {hire_rate_female:.3f}")
print(f"Hire rate Male:   {hire_rate_male:.3f}")
print(f"Parity gap:       {abs(hire_rate_male - hire_rate_female):.3f}")

# Mitigation: Remove protected attribute from features
X_fair = income.reshape(-1, 1)
model_fair = LogisticRegression()
model_fair.fit(X_fair, hired)
y_pred_fair = model_fair.predict(X_fair)

hire_fair_f = y_pred_fair[gender == 0].mean()
hire_fair_m = y_pred_fair[gender == 1].mean()
print(f"\nAfter mitigation - Parity gap: {abs(hire_fair_m - hire_fair_f):.3f}")
```

Listing 25: Detecting and Mitigating Bias

8.2.3 Misinformation Prevention

```python
from transformers import pipeline
```

```python

# Using a pre-trained detector
detector = pipeline("text-classification",
                    model="roberta-base-openai-detector")

texts = [
    "The sun rises in the east and sets in the west.",
    "Quantum entanglement proves that cats can simultaneously exist in Paris and Tokyo.",
]

for text in texts:
    result = detector(text)
    label = result[0]['label']
    score = result[0]['score']
    print(f"Text: {text[:50]}...")
    print(f"  -> {label} (confidence: {score:.3f})\n")

# Responsible AI checklist for LLM outputs
checklist = [
    "Does the output contain verifiable factual claims?",
    "Are sources cited where needed?",
    "Could this output be used to deceive or manipulate?",
    "Does it respect privacy of individuals?",
    "Is there potential for harmful downstream use?",
]
print("Pre-deployment checklist:")
for item in checklist:
    print(f"  [ ] {item}")
```

Listing 26: Detecting AI-Generated Text (Basic Approach)

9 Transformers (Deep Dive)

9.1 Architecture Overview

The complete Transformer architecture consists of an Encoder stack and Decoder stack (for seq2seq tasks), or Decoder-only (e.g., GPT) or Encoder-only (e.g., BERT).

```python
import torch
import torch.nn as nn
import math

class FeedForward(nn.Module):
    def __init__(self, d_model, d_ff, dropout=0.1):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(d_ff, d_model),
            nn.Dropout(dropout)
        )
    def forward(self, x): return self.net(x)

class TransformerBlock(nn.Module):
    def __init__(self, d_model=512, num_heads=8, d_ff=2048, dropout=0.1):
        super().__init__()
        self.attn   = nn.MultiheadAttention(d_model, num_heads,
                                            dropout=dropout, batch_first=True)
        self.ff     = FeedForward(d_model, d_ff, dropout)
        self.norm1  = nn.LayerNorm(d_model)
        self.norm2  = nn.LayerNorm(d_model)
        self.drop   = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        # Pre-LayerNorm (modern style)
        attn_out, _ = self.attn(self.norm1(x), self.norm1(x), self.norm1(x),
                                attn_mask=mask)
        x = x + self.drop(attn_out)          # residual connection
        x = x + self.drop(self.ff(self.norm2(x)))
        return x

class MiniGPT(nn.Module):
    def __init__(self, vocab_size, d_model=256, num_heads=8,
                 num_layers=4, d_ff=1024, max_seq=512, dropout=0.1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model)
        self.pos_emb   = nn.Embedding(max_seq, d_model)
        self.blocks    = nn.ModuleList(
            [TransformerBlock(d_model, num_heads, d_ff, dropout) for _ in range(num_layers)]
        )
        self.norm   = nn.LayerNorm(d_model)
        self.head   = nn.Linear(d_model, vocab_size, bias=False)

```

```python
    def forward(self, idx):
        B, T = idx.shape
        tok_emb = self.embedding(idx)
        pos = torch.arange(T, device=idx.device)
        pos_emb = self.pos_emb(pos)
        x = tok_emb + pos_emb
        # Causal mask (decoder-style)
        mask = torch.triu(torch.ones(T, T, device=idx.device), diagonal=1).bool()
        mask = mask.float().masked_fill(mask, float('-inf'))
        for block in self.blocks:
            x = block(x, mask)
        x = self.norm(x)
        return self.head(x)

gpt = MiniGPT(vocab_size=50000)
ids = torch.randint(0, 50000, (2, 64))
logits = gpt(ids)
print("Logits shape:", logits.shape)  # (2, 64, 50000)
```

Listing 27: Full Transformer Block

10 Generative AI & LLM Tooling Ecosystem

10.1 Overview

The modern GenAI ecosystem consists of multiple layers:

- **Foundation Models:** Pre-trained LLMs (GPT-4, Claude, Gemini, Llama 3, Mistral).
- **Fine-tuning Frameworks:** HuggingFace PEFT, Unsloth, Axolotl.
- **Orchestration:** LangChain, LlamaIndex, Haystack.
- **Vector Databases:** Pinecone, Weaviate, Chroma, Qdrant, FAISS.
- **Evaluation:** RAGAS, TruLens, DeepEval.
- **Deployment:** vLLM, Ollama, TGI (Text Generation Inference).

```python
from langchain_openai import ChatOpenAI
from langchain.schema import HumanMessage, SystemMessage
from langchain.prompts import ChatPromptTemplate
from langchain.chains import LLMChain
from langchain.memory import ConversationBufferMemory

# --- Basic Chat ---
llm = ChatOpenAI(model="gpt-4o", temperature=0.7)

messages = [
    SystemMessage(content="You are a helpful AI tutor for machine learning."),
    HumanMessage(content="Explain gradient descent in simple terms.")
]
response = llm.invoke(messages)
print(response.content)

# --- Prompt Templates ---
template = ChatPromptTemplate.from_messages([
    ("system", "You are an expert in {domain}."),
    ("human",  "Explain {concept} with a Python example.")
])
chain = template | llm
result = chain.invoke({"domain": "deep learning", "concept": "batch normalisation"})
print(result.content)

# --- Conversation with Memory ---
memory = ConversationBufferMemory(return_messages=True)
from langchain.chains import ConversationChain
convo = ConversationChain(llm=llm, memory=memory, verbose=False)
convo.predict(input="What is an embedding?")
convo.predict(input="How is it used in RAG systems?")
print(memory.buffer)
```

Listing 28: LangChain Ecosystem Basics

11 Commercial APIs

11.1 OpenAI API

```python
from openai import OpenAI

client = OpenAI(api_key="YOUR_API_KEY")

# --- Chat Completions ---
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system",  "content": "You are an expert ML engineer."},
        {"role": "user",    "content": "What is the vanishing gradient problem?"},
    ],
    temperature=0.7,
    max_tokens=500,
    top_p=1.0,
    frequency_penalty=0.0,
    presence_penalty=0.0
)
print(response.choices[0].message.content)
print(f"Tokens used: {response.usage.total_tokens}")

# --- Streaming ---
stream = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "Explain backpropagation step by step."}],
    stream=True
)
for chunk in stream:
    if chunk.choices[0].delta.content:
        print(chunk.choices[0].delta.content, end="", flush=True)

# --- Embeddings ---
emb_response = client.embeddings.create(
    model="text-embedding-3-small",
    input=["transformer architecture", "attention mechanism"]
)
embedding = emb_response.data[0].embedding
print(f"\nEmbedding dimension: {len(embedding)}")  # 1536
```

Listing 29: OpenAI Chat Completions API

11.2 Anthropic Claude API

```python
import anthropic

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

message = client.messages.create(
    model="claude-opus-4-5",
```

```
    max_tokens=1024,
    system="You are a concise AI tutor specialising in LLMs and GenAI.",
    messages=[
        {"role": "user", "content": "Explain the difference between RAG and fine-tuning."}
    ]
)
print(message.content[0].text)
print(f"Input tokens:  {message.usage.input_tokens}")
print(f"Output tokens: {message.usage.output_tokens}")
```

Listing 30: Anthropic Claude API

12 Prompt Engineering and Fine-Tuning

12.1 Prompt Engineering

Key Concept

Prompt engineering is the practice of designing inputs to LLMs to elicit the best possible outputs. It is the fastest and cheapest way to adapt LLM behaviour.

```python
from openai import OpenAI
client = OpenAI()

# 1. Zero-Shot Prompting
zero_shot = """
Classify the following text as POSITIVE, NEGATIVE, or NEUTRAL.
Text: "The new product launch was underwhelming and failed to meet expectations."
Classification:
"""

# 2. Few-Shot Prompting
few_shot = """
Classify sentiment. Examples:
Text: "I love this!"            -> POSITIVE
Text: "This is terrible."       -> NEGATIVE
Text: "It arrived on time."     -> NEUTRAL

Text: "The battery life is disappointing but the screen is great."
Classification:
"""

# 3. Chain-of-Thought (CoT)
cot = """
Solve step by step:
A model achieves 80% accuracy on training data and 60% on test data.
What is happening and how would you fix it?

Think step by step:
"""

# 4. Role Prompting
role = """
You are a senior data scientist with 10 years of experience.
Critique the following approach and suggest improvements:
"I'm using accuracy as my only metric for an imbalanced dataset with 95% negative class."
"""

# 5. Structured Output Prompting
structured = """
Analyse the following model and return a JSON response:
Model: Decision Tree with depth=50, trained on 1000 samples, test accuracy=0.61

Return ONLY valid JSON with keys: issue, severity (high/medium/low), recommendation
```

```python
"""

def ask(prompt, model="gpt-4o"):
    r = client.chat.completions.create(
        model=model,
        messages=[{"role":"user","content":prompt}],
        temperature=0.2
    )
    return r.choices[0].message.content

print(ask(few_shot))
```

Listing 31: Prompt Engineering Techniques

12.2 Fine-Tuning with Parameter-Efficient Methods (LoRA)

Formula

LoRA (Low-Rank Adaptation): Instead of updating all weights W, decompose the update as:

$$W' = W + \Delta W = W + BA$$

Where $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$, and $r \ll \min(d, k)$.
Only A and B are trained, reducing trainable parameters by $> 99\%$.

```python
from transformers import AutoModelForCausalLM, AutoTokenizer,
    TrainingArguments, Trainer
from peft import get_peft_model, LoraConfig, TaskType
from datasets import load_dataset
import torch

model_name = "facebook/opt-125m"
tokenizer  = AutoTokenizer.from_pretrained(model_name)
model      = AutoModelForCausalLM.from_pretrained(model_name,
    torch_dtype=torch.float16)

# Configure LoRA
lora_config = LoraConfig(
    task_type      = TaskType.CAUSAL_LM,
    r              = 8,              # rank
    lora_alpha     = 32,             # scaling
    target_modules = ["q_proj", "v_proj"],  # which layers to adapt
    lora_dropout   = 0.05,
    bias           = "none",
)
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# Trainable params: ~0.1% of total

# Training
training_args = TrainingArguments(
    output_dir          = "./lora-finetuned",
    num_train_epochs    = 3,
    per_device_train_batch_size = 4,
    learning_rate       = 2e-4,
```

```
    fp16                    = True,
    save_steps              = 100,
    logging_steps           = 50,
    warmup_ratio            = 0.05,
)

# Dataset preparation
def tokenize(examples):
    return tokenizer(examples["text"], truncation=True, max_length
    =512)

dataset = load_dataset("wikitext", "wikitext-2-raw-v1", split="train
    [:1%]")
dataset = dataset.map(tokenize, batched=True, remove_columns=["text"
    ])

trainer = Trainer(model=model, args=training_args, train_dataset=
    dataset)
trainer.train()
```

Listing 32: Fine-Tuning with LoRA using PEFT

13 RAG Agents: Pipelines, Embeddings & Vector Databases

13.1 What is RAG?

Retrieval-Augmented Generation (RAG) enhances LLM responses by retrieving relevant documents from a knowledge base at inference time. This mitigates hallucinations and keeps knowledge current without full retraining.

RAG Pipeline:

$$\text{Query} \xrightarrow{\text{Embed}} \vec{q} \xrightarrow{\text{Similarity Search}} \text{Top-}k\text{ Docs} \xrightarrow{\text{Prompt}} \text{LLM} \rightarrow \text{Answer}$$

13.2 Embedding Models

```python
from sentence_transformers import SentenceTransformer
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

# Load embedding model
model = SentenceTransformer("all-MiniLM-L6-v2")  # 384-dim, fast

# Corpus (knowledge base)
documents = [
    "Transformers use self-attention to process sequences in parallel.",
    "RAG retrieves relevant documents to augment LLM prompts.",
    "Fine-tuning adapts a pre-trained model to a specific task.",
    "Vector databases store embeddings for efficient similarity search.",
    "RLHF aligns LLMs with human preferences using reward models.",
]

# Encode all documents
doc_embeddings = model.encode(documents, normalize_embeddings=True)
print("Document embeddings shape:", doc_embeddings.shape)  # (5, 384)

# Query
query = "How does retrieval augmented generation work?"
query_emb = model.encode([query], normalize_embeddings=True)

# Cosine similarity search
similarities = cosine_similarity(query_emb, doc_embeddings)[0]
top_k = np.argsort(similarities)[::-1][:2]  # top-2

print(f"\nQuery: {query}")
print("Top retrieved documents:")
for idx in top_k:
    print(f"  [{similarities[idx]:.4f}] {documents[idx]}")
```

Listing 33: Generating Embeddings for RAG

13.3 Vector Databases

```python
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")

documents = [
    "The attention mechanism allows the model to focus on relevant parts.",
    "BERT is an encoder-only transformer pre-trained with masked language modelling.",
    "GPT uses a decoder-only transformer for autoregressive text generation.",
    "Embeddings are dense vector representations of text in semantic space.",
    "Vector similarity search finds semantically related documents efficiently.",
    "RAG combines retrieval systems with generative models.",
]

# Create embeddings
embeddings = model.encode(documents)
embeddings = np.array(embeddings, dtype='float32')
d = embeddings.shape[1]  # dimension (384)

# Build FAISS index
faiss.normalize_L2(embeddings)
index = faiss.IndexFlatIP(d)       # Inner Product (cosine after normalisation)
index.add(embeddings)
print(f"Index contains {index.ntotal} vectors")

# Search
query = "How does GPT generate text?"
query_emb = model.encode([query]).astype('float32')
faiss.normalize_L2(query_emb)

k = 3
distances, indices = index.search(query_emb, k)
print(f"\nQuery: {query}")
for i, (dist, idx) in enumerate(zip(distances[0], indices[0])):
    print(f"  Rank {i+1} [score={dist:.4f}]: {documents[idx]}")
```

Listing 34: Using FAISS as Vector Database

13.4 Complete RAG Pipeline

```python
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer
from openai import OpenAI

# --- Step 1: Build Knowledge Base ---
documents = [
```

```
    "Transformers process all tokens in parallel using self-attention mechanisms.",
    "BERT is pre-trained using Masked Language Modelling (MLM) and NSP tasks.",
    "GPT models use causal (left-to-right) attention for autoregressive generation.",
    "RAG stands for Retrieval Augmented Generation.",
    "LoRA reduces trainable parameters by adding low-rank decomposition matrices.",
    "Vector databases like Pinecone, Chroma, and FAISS store dense embeddings.",
    "Hallucination in LLMs refers to generating plausible but factually incorrect content.",
    "Fine-tuning adapts a pre-trained model on task-specific data.",
]

embed_model = SentenceTransformer("all-MiniLM-L6-v2")
embeddings = embed_model.encode(documents).astype('float32')
faiss.normalize_L2(embeddings)

index = faiss.IndexFlatIP(embeddings.shape[1])
index.add(embeddings)

# --- Step 2: Retrieval ---
def retrieve(query: str, top_k: int = 3):
    q_emb = embed_model.encode([query]).astype('float32')
    faiss.normalize_L2(q_emb)
    distances, indices = index.search(q_emb, top_k)
    return [documents[i] for i in indices[0]]

# --- Step 3: Augmented Generation ---
def rag_answer(question: str, top_k: int = 3):
    context_docs = retrieve(question, top_k)
    context = "\n".join([f"- {doc}" for doc in context_docs])

    client = OpenAI()
    prompt = f"""You are a helpful AI assistant. Answer the question using ONLY
the provided context. If the answer is not in the context, say 'I don't know.'

Context:
{context}

Question: {question}
Answer:"""

    response = client.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.1,
        max_tokens=300
    )
    return response.choices[0].message.content, context_docs

# Run
answer, sources = rag_answer("What is LoRA and how does it help fine-tuning?")
```

```python
print("Answer:", answer)
print("\nSources used:")
for s in sources: print(f"  - {s}")
```

Listing 35: End-to-End RAG Pipeline

14 Responsible Use of AI

14.1 Core Principles

Responsible AI is an approach to developing and deploying AI systems that prioritises ethical, social, and legal considerations alongside technical performance.

- **Transparency & Explainability:** Users should understand why a system made a decision.
- **Fairness:** Systems should not perpetuate or amplify societal biases.
- **Privacy:** Minimal collection and proper protection of personal data.
- **Safety:** Preventing harm from AI outputs, including misinformation and manipulation.
- **Accountability:** Clear responsibility for AI-driven outcomes.
- **Environmental Impact:** Awareness of the carbon footprint of large model training.

14.2 Explainability with SHAP and LIME

```python
import shap
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

# Train model
data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)
clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)

# SHAP Explainer
explainer = shap.TreeExplainer(clf)
shap_values = explainer.shap_values(X_test)

# Feature importance (global)
print("SHAP Feature Importances:")
mean_shap = np.abs(shap_values[1]).mean(axis=0)
for name, imp in sorted(zip(data.feature_names, mean_shap),
                        key=lambda x: x[1], reverse=True)[:5]:
    print(f"  {name:<30}: {imp:.4f}")

# Local explanation for one sample
print(f"\nPredicted class for sample 0: {clf.predict(X_test[:1])[0]}"
    )
print("Top positive SHAP values (pushes toward class 1):")
sample_shap = shap_values[1][0]
top_idx = np.argsort(sample_shap)[::-1][:3]
```

```python
for i in top_idx:
    print(f"  {data.feature_names[i]:<30}: {sample_shap[i]:+.4f}")
```

Listing 36: Model Explainability with SHAP

14.3 Data Privacy and Differential Privacy

```python
# Differential Privacy using PyTorch Opacus
# pip install opacus

import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
from opacus import PrivacyEngine

# Simple model
model = nn.Sequential(
    nn.Linear(20, 64), nn.ReLU(),
    nn.Linear(64, 32), nn.ReLU(),
    nn.Linear(32, 1),  nn.Sigmoid()
)

optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

# Dummy dataset
X = torch.randn(1000, 20)
y = torch.randint(0, 2, (1000, 1)).float()
dataset = TensorDataset(X, y)
loader  = DataLoader(dataset, batch_size=64, shuffle=True)

# Attach PrivacyEngine
privacy_engine = PrivacyEngine()
model, optimizer, loader = privacy_engine.make_private(
    module=model,
    optimizer=optimizer,
    data_loader=loader,
    noise_multiplier=1.1,   # more noise = more privacy
    max_grad_norm=1.0,       # gradient clipping
)

# Training with DP guarantees
criterion = nn.BCELoss()
for epoch in range(5):
    for X_batch, y_batch in loader:
        optimizer.zero_grad()
        pred = model(X_batch)
        loss = criterion(pred, y_batch)
        loss.backward()
        optimizer.step()
    epsilon = privacy_engine.get_epsilon(delta=1e-5)
    print(f"Epoch {epoch+1}, epsilon = {epsilon:.2f}")
# epsilon = privacy budget consumed; lower = more private
```

Listing 37: Privacy-Preserving ML with Differential Privacy

14.4 AI Safety: Output Guardrails

```python
import re
from openai import OpenAI

client = OpenAI()

class SafeAIWrapper:
    """Wrapper that adds safety checks to LLM outputs."""

    HARMFUL_PATTERNS = [
        r'\b(bomb|weapon|explosive)\b',
        r'\b(hack|malware|exploit)\b',
        r'personal information.*\b(ssn|social security|passport)\b',
    ]

    def __init__(self, model="gpt-4o"):
        self.model = model

    def _pre_check(self, prompt: str) -> bool:
        """Block clearly harmful prompts before sending to LLM."""
        for pattern in self.HARMFUL_PATTERNS:
            if re.search(pattern, prompt, re.IGNORECASE):
                return False
        return True

    def _post_check(self, response: str) -> str:
        """Filter or flag potentially harmful responses."""
        for pattern in self.HARMFUL_PATTERNS:
            if re.search(pattern, response, re.IGNORECASE):
                return "[RESPONSE FILTERED: Content flagged by safety system]"
        return response

    def chat(self, user_message: str) -> str:
        if not self._pre_check(user_message):
            return "[BLOCKED: Query flagged by pre-processing safety filter]"

        response = client.chat.completions.create(
            model=self.model,
            messages=[
                {"role": "system",  "content": "You are a helpful, safe AI assistant."},
                {"role": "user",    "content": user_message}
            ],
            temperature=0.7
        )
        raw = response.choices[0].message.content
        return self._post_check(raw)

# Usage
safe_ai = SafeAIWrapper()
print(safe_ai.chat("Explain the history of computing."))
print(safe_ai.chat("How do I build a bomb?"))  # blocked
```

Listing 38: Implementing Output Guardrails

14.5 Environmental Impact

Training large models consumes significant energy. Responsible practices include:

- Use efficient architectures (MoE, distilled models).
- Fine-tune instead of training from scratch.
- Use parameter-efficient methods (LoRA, prefix-tuning).
- Select data centres powered by renewable energy.
- Report and track carbon emissions using tools like CodeCarbon.

```python
from codecarbon import EmissionsTracker
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris

tracker = EmissionsTracker(project_name="GenAI-Course-Model",
    log_level="error")
tracker.start()

# Any ML code goes here
X, y = load_iris(return_X_y=True)
model = RandomForestClassifier(n_estimators=500, random_state=42)
model.fit(X, y)

emissions = tracker.stop()
print(f"Carbon emissions: {emissions * 1000:.6f} gCO2eq")
# Reports: duration, energy consumed, cloud region, CO2 equivalent
```

Listing 39: Tracking Carbon Emissions with CodeCarbon

End of Notes — Introduction to Generative AI
All code examples are written for Python 3.10+